

Nomenclature:-

- df_d :- data type read by system./Infered data type of columns
- d :- dataframe for data dictionary hosted on website
- merged_df :- combined dataframes,"d" and "df_d".
- .copy() :- used as checkpoint to store the work done for caution.
- unique_values_dict:- dictionary to store, 'columns' as key and their top 5 'unique_values' as value
- inmerged :- dataframe containing columns with incorrect datatypes.

```
In [ ]: import pandas as pd
        from huggingface_hub import hf_hub_download
```

```
%matplotlib inline
```

```
pd.set_option('display.max_colwidth',None)
pd.set_option('display.max_rows',50)
from warnings import filterwarnings
filterwarnings('ignore')
```

```
In [8]: repo_id = "zafar0171/loan_default_dataset"
        file1 = 'loan_data_2007_2014.csv'
        file2 = 'loan_data_2015.csv'

        path1 = hf_hub_download(repo_id=repo_id, filename =file1, repo_type ="dataset")
        path2 = hf_hub_download(repo_id = repo_id, filename=file2, repo_type = "dataset")

        ld1 = pd.read_csv(path1,low_memory = False)
        ld2 = pd.read_csv(path2, low_memory = False)

        df = pd.concat([ld1, ld2])
        df.reset_index(drop =True, inplace = True)

        df.loan_status.unique().tolist()
        df.loan_status.value_counts()

        df.drop(df[df['loan_status'].isin(['Current','Charged Off',
            'Late (31-120 days)', 'In Grace Period', 'Late (16-30 days)',
            'Does not meet the credit policy. Status:Fully Paid',
            'Does not meet the credit policy. Status:Charged Off', 'Issued'])].index,inplace = True)

        df.reset_index(drop=True, inplace = True)
        df.loan_status.unique()
```

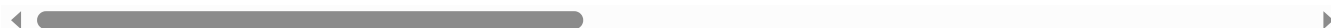
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.

```
Out[8]: array(['Fully Paid', 'Default'], dtype=object)
```

```
In [9]: ld1.head(3)
```

Out[9]:	Unnamed: 0	id	member_id	loan_amnt	funded_amnt	funded_amnt_inv	term	int_rate	inst
0	0	1077501	1296599	5000	5000	4975.0	36 months	10.65	
1	1	1077430	1314167	2500	2500	2500.0	60 months	15.27	
2	2	1077175	1313524	2400	2400	2400.0	36 months	15.96	

3 rows × 75 columns

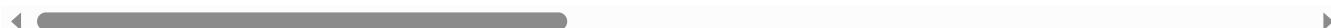


In [10]: `ld2.tail(3)`

Out[10]:

	id	member_id	loan_amnt	funded_amnt	funded_amnt_inv	term	int_rate	installment
421091	36271333	38982739	13000	13000	13000.0	60 months	15.99	316.0
421092	36490806	39222577	12000	12000	12000.0	60 months	19.99	317.0
421093	36271262	38982659	20000	20000	20000.0	36 months	11.99	664.0

3 rows × 74 columns



x

In [29]: `df.shape`

Out[29]: (208942, 54)

In [31]: *###Note:- Missing vals are being Imputed.*

```

mv = pd.DataFrame({'mv_count':df.isnull().sum(),
                   'percent_missing':df.isnull().sum()/len(df) *100}
                  ).sort_values(by = ['percent_missing'], ascending = False)
mv.head(25)

print('1)-----Total no of columns having null values more than 55% are: ',len(mv[mv['per
'\n', ' ', 'total number of columns in dataset:, \n\n {} '.format(df.shape[1]))

"""
21 columns in our dataset has more than 56% of missing values present in them, and out of 21 c
only 1 column has less than 81% missing values.
So we are going to drop all the columns having more than 56% of values present in them.
"""

df.drop(mv[mv['percent_missing'].gt(56)].index, axis = 1, inplace = True)
new= df.copy()

new_mv = pd.DataFrame({'mv_count':df.isnull().sum(),
                       'percent_missing':df.isnull().sum()/len(df) *100}
                      ).sort_values(by = ['percent_missing'], ascending = False).head
new_mv.head(12)

```

```

for row in ['title', 'emp_length', 'last_pymnt_d', 'last_credit_pull_d']:
    df[row].fillna(df[row].mode()[0], inplace = True)
for row in new_mv.index[:12]:
    if df[row].dtypes in ['float64', 'int64']:
        df[row].fillna(df[row].median(), inplace = True)
    else:
        pass
df.shape

#Imputation done
# -----

#creating a dataframe having variables and their datatypes in seperate columns

df_d = pd.DataFrame(df.dtypes)          #dataframe for datatypes as read by system
df_d.reset_index(inplace = True)
print("2)-----columns data type infered by system, \n\n {}".format(df_d.head(5)))

##### importing original csv file containing variables and thier actual datatypes along with their

#importing original csv file containing variables and thier actual datatypes along with their
d = pd.read_excel('lclub.xlsx', index_col = 0)    #d ---- dataframe for data dictionary hosted
print("3)-----columns with their actual data types, \n\n {}".format(d.head()))
#concatenating
merged_df = df_d.merge(d, left_on='index', right_on='index', how='inner')

print("4)-----dataframe having columns and their original and system infered datat types")
print("5)-----merged dataframe shape, \n\n {}".format(merged_df.shape))

#changing the column name

merged_df.rename(columns = {0 : 'env_dtype'}, inplace = True)
print("6)-----changed column name '0' to env_dtype, \n\n {}".format(merged_df.head()))

for col in merged_df[['env_dtype', 'data_types']]:
    print("7)-----unique values present in :, \n {} \n {}".format(col, merged_df[col].unique()))

merged_df['data_types'].replace(['obj', 'obj/cat', 'categorical'], ['object', 'object', 'object'],

cd_n = merged_df.copy()
#print("cd_n.head()")

print("8)-----Total column having correct datatypes are:, \n\n {}".format(len(merged_df[merged_df['data_types'] == 'object'])))
print("9)-----columns having correct data types, \n\n {}".format(merged_df[merged_df['data_types'] == 'object'].shape[0]))

### Following variables have incorrect datatype, we will convert the data into their desired datatype

print("10)-----Total number of column having incorrect datatypes are:, \n\n {}".format(len(merged_df[merged_df['data_types'] != 'object'])))
print("11)-----columns having INcorrect data types, \n\n {}".format(merged_df[merged_df['data_types'] != 'object'].shape[0]))

# creating a dictionary, having 'variable' as key and their top 5 'unique_values' as value,

```

```
# to use this dictionary in our datatype dataframe
```

```
unique_values_dict = {}  
for i in df.columns:  
    unique_values = list(pd.Series(df[i].unique()))  
    uv = unique_values[:5] #uv - unique_values  
    #lis1.append(unique_values[:5])  
    unique_values_dict[i] = uv
```

```
#creating a new column corresponging to the variables present in the index column
```

```
merged_df['uniq_va'] = merged_df['index'].map(unique_values_dict)
```

```
print("12)-----unique val column containing unique values of dataset variables, \n")
```

```
##### creating a new datfarama using 'datatype' datfarama for column having 'incorrect datatype'
```

```
inmerged_df = merged_df[~merged_df['data_types'].eq(merged_df['env_dtype'])]  
inmerged_df.head()
```

```
inmerged_df[inmerged_df['data_types'].eq('yearmonth')]
```

```
#Data type correction
```

```
for col in ['issue_d', 'earliest_cr_line', 'last_pymnt_d', 'last_credit_pull_d']:  
    df[col] = pd.to_datetime(df[col], format='%b-%y', errors='coerce')
```

```
df1 = df.copy()
```

```
for i in inmerged_df['index']:
```

```
    data_type_value = inmerged_df.loc[inmerged_df['index'] == i, 'data_types'].iloc[0]
```

```
    if data_type_value in ['int64', 'object']:
```

```
        if data_type_value in ['int64']:  
            df[i] = df[i].astype(data_type_value)
```

```
        else:  
            df[i] = df[i].astype(data_type_value)
```

```
    else:
```

```
        print(f"Invalid data type value '{data_type_value}' for index '{i}'. Skipping conversion")  
        continue
```

1)-----Total no of columns having null values more than 55% are: 21
total number of columns in dataset:,

75

2)-----columns data type infered by system,

	index	0
0	Unnamed: 0	float64
1	id	int64
2	member_id	int64
3	loan_amnt	int64
4	funded_amnt	int64

3)-----columns with their actual data types,

	index	data_types	\
0	id	obj	
1	member_id	obj	
2	loan_amnt	int64	
3	funded_amnt	int64	
4	funded_amnt_inv	float64	

	description
0	A unique LC assigned ID for the loan listing.
1	A unique LC assigned ID for the borrower member.
2	The listed amount of the loan applied for by t...
3	The total amount committed to that loan at tha...
4	The total amount committed by investors for th...

4)-----dataframe having columns and their original and system infered datat types

5)-----merged dataframe shape,

(53, 4)

6)-----changed column name '0' to env_dtype,

	index	env_dtype	data_types	\
0	id	int64	obj	
1	member_id	int64	obj	
2	loan_amnt	int64	int64	
3	funded_amnt	int64	int64	
4	funded_amnt_inv	float64	float64	

	description
0	A unique LC assigned ID for the loan listing.
1	A unique LC assigned ID for the borrower member.
2	The listed amount of the loan applied for by t...
3	The total amount committed to that loan at tha...
4	The total amount committed by investors for th...

7)-----unique values present in :,

env_dtype

[dtype('int64') dtype('float64') dtype('O')]

7)-----unique values present in :,

data_types

['obj' 'int64' 'float64' 'obj/cat' 'yearmonth' 'object' 'categorical']

8)-----Total column having correct datatypes are:,

38

9)-----columns having correct data types,

	index	env_dtype	data_types	\
2	loan_amnt	int64	int64	
3	funded_amnt	int64	int64	
4	funded_amnt_inv	float64	float64	
5	term	object	object	
6	int_rate	float64	float64	

description

2 The listed amount of the loan applied for by t...
 3 The total amount committed to that loan at tha...
 4 The total amount committed by investors for th...
 5 The number of payments on the loan. Values are...
 6 The interest rate on the loan.
 10)-----Total number of column having incorrect datatypes are:,

15

11)-----columns having INcorrect data types,

	index	env_dtype	data_types \
0	id	int64	object
1	member_id	int64	object
15	issue_d	object	yearmonth
24	delinq_2yrs	float64	int64
25	earliest_cr_line	object	yearmonth
26	inq_last_6mths	float64	object
27	mths_since_last_delinq	float64	object
28	open_acc	float64	int64
29	pub_rec	float64	int64
32	total_acc	float64	int64
43	last_pymnt_d	object	yearmonth
45	last_credit_pull_d	object	yearmonth
46	collections_12_mths_ex_med	float64	int64
47	policy_code	int64	object
49	acc_now_delinq	float64	int64

	description
0	A unique LC assigned ID for the loan listing.
1	A unique LC assigned ID for the borrower member.
15	The month which the loan was funded.
24	The number of 30+ days past-due incidences of ...
25	The month the borrower's earliest reported cre...
26	The number of inquiries in the past 6 months (...)
27	The number of months since the borrower's last...
28	The number of open credit lines in the borrowe...
29	The number of derogatory public records.
32	The total number of credit lines currently in ...
43	Last month payment was received.
45	The most recent month LC pulled credit for thi...
46	Number of collections in 12 months excluding m...
47	Publicly available policy_code=1; new products...
49	The number of accounts on which the borrower i...

12)-----unique val column containing unique values of dataset variables,

	index	env_dtype	data_types \
39	total_rec_int	float64	float64
40	total_rec_late_fee	float64	float64
28	open_acc	float64	int64
32	total_acc	float64	int64
11	emp_length	object	object

	description \
39	Interest received to date.
40	Late fees received to date.
28	The number of open credit lines in the borrowe...
32	The total number of credit lines currently in ...
11	Employment length in years. Possible values ar...

	uniq_va
39	[861.07, 603.65, 2209.33, 631.38, 938.14]
40	[0.0, 16.97, 15.0, 14.98883533, 23.78149961]
28	[3.0, 2.0, 10.0, 9.0, 4.0]
32	[9.0, 10.0, 37.0, 12.0, 4.0]
11	[10+ years, 3 years, 9 years, 5 years, < 1 year]

Invalid data type value 'yearmonth' for index 'issue_d'. Skipping conversion.
Invalid data type value 'yearmonth' for index 'earliest_cr_line'. Skipping conversion.
Invalid data type value 'yearmonth' for index 'last_pymnt_d'. Skipping conversion.
Invalid data type value 'yearmonth' for index 'last_credit_pull_d'. Skipping conversion.

Converting target column into encoded form

```
In [ ]: df['loan_status'].replace({'Fully Paid':0,'Default':1},inplace = True)
```